

창작 과정 증명 서비스

Trace

팀원 작업 가이드 · Windows

창의프로젝트1 · 2026-2학기 · 박재완 교수 · 2인 1팀

이 문서 하나만 따라 하면 환경 설치부터 발표 준비까지 됩니다. 명령어는 한 줄씩 나눠 뒀으니 위에서부터 차례대로 복사해서 붙여 넣으세요.

처음이라면 여기부터

① GitHub 초대 수락 → ② 프로그램 두 개 설치 → ③ 환경 만들기 → ④ 실행해서 화면 확인 → ⑤ CHANGES.md 읽기 → ⑥ 아래 4장 「내가 할 일」.

막히면 바로 A에게 연락하세요. 혼자 30분 이상 붙잡지 마세요.

A의 컴퓨터는 맥입니다

A가 보내 주는 명령어에 `cp` 나 `~/dev` 같은 게 섞여 있으면 맥 명령어입니다. 윈도우에서는 안 됩니다. 이 문서에 적힌 쪽을 쓰세요.

코드 자체(`python app.py`, `git` 명령어)는 양쪽이 똑같습니다. 다른 것은 폴더 만들기·파일 복사 정도입니다.

한눈에 보기

내용

이 서비스가 하는 일	창작 과정을 기록해 두고, 그 기록이 나중에 고쳐지지 않았음을 제3자에게 보여준다
하지 않는 일	AI 사용 여부를 판정하지 않는다. 지표는 자료이지 결론이 아니다
이번 주 과제 요건	한 페이지 구현 · JSON 데이터가 화면에 나타날 것 · 데이터 흐름에 집중
기술 범위	HTML/CSS/JS + Flask + JSON 파일 (강의자료 STEP 1). React·서버DB·배포는 범위 밖
저장소	github.com/20233014-KYH/trace

1. 환경 설치

① GitHub 초대를 먼저 수락하세요

초대장이 메일이나 GitHub 알림으로 갑니다. 안 보이면 아래 주소로 직접 들어가서 Accept invitation 을 누르세요. **7일 뒤 만료됩니다.**

```
https://github.com/20233014-KYH/trace/invitations
```

② 프로그램 두 개를 설치합니다

프로그램

어디서 · 무엇을 고르나

Anaconda	anaconda.com/download — Windows 64-Bit Graphical Installer. 설치 중 선택지는 전부 기본값으로 넘기면 됩니다
----------	--

Git	git-scm.com/download/win — 64-bit Setup. 이것도 전부 기본값
-----	---

VS Code	code.visualstudio.com — 필수는 아니지만 뒤에서 chain.json 을 열 때 필요합니다
---------	---

③ Anaconda Prompt 를 엽니다

일반 명령 프롬프트(cmd)나 PowerShell 이 아닙니다. 시작 메뉴를 누르고 Anaconda Prompt 라고 치면 나옵니다. 앞으로 나오는 명령어는 전부 이 검은 창에 칩니다.

④ 파이썬 환경을 만듭니다

```
conda create -n cp python=3.11 -y
```

```
conda activate cp
```

성공하면 줄 맨 앞이 (base) 에서 (cp) 로 바뀝니다.

```
pip install flask
```

⑤ 코드를 받습니다

```
cd %USERPROFILE%
```

```
mkdir dev
```

이미 있다고 나오면 그냥 넘어가면 됩니다.

```
cd dev
```

```
git clone https://github.com/20233014-KYH/trace.git
```

```
cd trace
```

⑥ 데모 데이터를 만듭니다

```
python seed.py
```

이걸 먼저 돌려야 화면에 뭔가 나옵니다. data 폴더는 GitHub 에 올리지 않기 때문입니다 (각자 컴퓨터에서 계속 바뀌는 파일이라 같이 쓰면 매번 충돌이 납니다).

⑦ 서버를 켭니다

```
python app.py
```

브라우저 주소창에 127.0.0.1:5001 을 칩니다. 끄고 싶으면 그 검은 창에서 Ctrl + C 입니다.

윈도우에서 자주 걸리는 것 세 가지

방화벽 팝업 — 처음 실행하면 “Windows Defender 방화벽에서 이 앱의 일부 기능을 차단했습니다”가 뜹니다. 액세스 허용을 누르세요. 취소해도 127.0.0.1 은 열리지만 팝업이 계속 뜹니다.

창을 두 개 쓰세요 — 서버를 켜는 창에는 명령어를 칠 수 없습니다. 시작 메뉴에서 Anaconda Prompt 를 하나 더 열어서 쓰세요. “서버 켜는 창 1개 + 명령어 치는 창 1개”로 나눠 두면 편합니다.

한글이 깨져 보이면 — 검은 창에 chcp 65001 을 한 번 치고 다시 실행하세요.

2. 잘 됐는지 확인

화면에서 이 다섯 가지가 보이면 정상입니다.

1 맨 위에 초록 상자 — “이 기록은 변조되지 않았습니다”

빨강이면 chain.json 이 고쳐진 것입니다. python seed.py 를 다시 돌리세요.

2 데모 1과 데모 2의 곡선이 확연히 다른가

데모 1은 막대가 오르내리고, 데모 2는 한 번 크게 튼 뒤 평평합니다. 이게 이 서비스의 핵심 장면입니다.

3 데모 3 카드에 파일 띠가 세 줄

러프 1시간 12분 / 선화 2시간 38분 / 채색 4시간 8분, 합계 7시간 58분.

4 러프.procreate 이름 앞에 주황 점

밖에서 이미지가 1장 들어왔다는 표시입니다.

5 작품 루트 해시 64자리

증명서에 찍히고 외부 앵커에 고정될 값입니다.

빨간 글씨가 떠도 고장이 아닙니다

WARNING: This is a development server — “연습용 서버지 실제 서비스용은 아니다”라는 안내입니다. 과제 범위에선 정상입니다.

GET /favicon.ico 404 — 브라우저가 탭 아이콘을 찾다가 없어서 나는 것입니다. 무시하세요.

3. 최근에 크게 바뀐 것 — 작품 계층

기록의 단위가 세션에서 작품으로 올라갔습니다. 증명서는 이제 파일 하나가 아니라 작품 하나에 한 장 나옵니다.

커미션 한 건이 러프 · 선화 · 채색 세 파일로 나뉘는데, 파일마다 증명서가 따로 나오면 의뢰인이 세 장을 받고 “이게 다 같은 그림 맞나요?”를 되물어야 하기 때문입니다.

데이터 구조

works > files > sessions > snapshots

층 무엇인가

작품 창작물 하나. 증명서는 여기에 한 장

파일 그 작품을 만들며 건드린 파일들 (러프 / 선화 / 채색 ...)

세션 한 번 앉아서 작업한 구간

스냅샷 30초마다 남긴 해시 한 줄. 여기 안쪽은 하나도 안 바뀌었습니다

파일을 묶는 방법 — 작업 중에는 묻지 않습니다

“이 파일은 이 작품 건가요?” 팝업을 매번 띄우면 사흘이면 반사적으로 누르게 되고, 그러면 그 동의가 아무것도 증명하지 못합니다. 그래서 세 겹으로 거릅니다.

① 기록 시작에서 볼 앱을 먼저 고른다 (브라우저·메신저는 기본 제외) → ② 그 안에서 열린 파일은 조용히 쌓인다 → ③ 종료할 때 한 번만 확인한다.

뺨 파일은 이름도 내용도 공개하지 않지만, “뺨다”는 사실과 시각은 기록에 남습니다. 공백 구간을 지우지 않고 빗금으로 남기는 것과 같은 원칙입니다.

자세한 내용은 저장소의 CHANGES.md 에 있습니다. 받아마자 한 번 읽어 주세요.

데모 세 개가 무엇을 보여주는가

작품	시간	붙여넣기	되돌리기	곡선
데모 1 — 처음부터 쓴 글	1시간 1분	0%	71회	오르내림
데모 2 — 붙여넣기가 많은 글	5분	95%	0회	거의 직선
데모 3 — 파일 세 개짜리 그림	7시간 58분	0%	362회	파일 3개

데모 1과 2는 같은 에세이.docx 입니다. 데모 2가 글자는 더 많은데 시간은 1/12 입니다. 기획안 6.2의 그림 두 개(탐색이 있는 과정 / 없는 과정)를 데이터로 만든 것입니다.

제목에 “AI”라는 말은 일부러 넣지 않았습니다. “붙여넣기가 많다”는 관찰된 사실이지만 “AI로 썼다”는 판정이기 때문입니다.

4. 내가 할 일 (B · 검증 · 기획 담당)

기획안 11.3 역할 분담 기준입니다. 위에서부터 급한 순서입니다.

① 데모 두 개를 에디터로 직접 만들기 — 가장 급함

지금 화면에 있는 데모는 **전부 프로그램이 만든 가짜 숫자**입니다. 발표에서 “이거 실제로 쓴 건가요?”라고 물으면 답할 수 없습니다. B가 직접 써서 진짜 기록으로 바꿔야 합니다.

절차

1 화면 아래 “작성 영역 열기”를 누른다

2 작품 이름에 **직접 쓴 글**, 파일 이름에 **에세이.docx** 를 넣고 “기록 시작”

20분 동안 아무 주제로 직접 씁니다. 고쳐 쓰고 지우고 되돌리기(Ctrl+Z)도 실제로 하세요. 그 흔적이 곡선이 됩니다.

3 “기록 종료”를 누른다

4 작품 이름을 **붙여넣은 글** 로 바꾸고 다시 “기록 시작”

다른 문서에서 1,000자쯤을 통째로 붙여 넣고 2~3분만에 끝냅니다.

5 두 작품의 곡선이 눈에 띄게 다른지 확인한다

다르지 않으면 다시 하세요. 이 두 장면이 발표의 핵심입니다.

만들고 나면 반드시 사본을 쓰세요

data 폴더는 GitHub 에 올라가지 않습니다. 직접 만든 기록은 그 컴퓨터에만 있고, 한 번 날아가면 **다시 만들 수 없습니다**. 아래 명령어로 사본을 떠 두고, USB 나 메일로도 한 부 챙겨 두세요.

```
copy data\chain.json data\chain.demo-good.json
```

망가뜨렸을 때 되돌리기

```
copy /Y data\chain.demo-good.json data\chain.json
```

② 코드 흐름을 설명할 수 있게 되기

기획안 11.4 기준입니다. “코드를 받아 쓰더라도 왜 그렇게 동작하는지 설명할 수 있어야 한다.” B가 맡은 부분은 static/script.js 입니다.

설명할 것

어디를 보면 되나

이벤트를 어떻게 세는가	editor 의 paste · keydown · input 핸들러. 붙여넣기는 input 보다 먼저 일어나서 따로 잡는다
--------------	---

왜 events 와 totals 를 나눴나	events 는 지난 30초의 변화량이라 서버로 보내고 비운다. totals 는 세션 누적이라 화면에 계속 쌓인다
-------------------------	---

화면에 어떻게 그려지는가	verify() → fetch('/api/verify') → renderVerdict / renderOverview / renderWorks
---------------	--

왜 프런트가 계산하지 않나	판단 재료를 만드는 것은 서버(app.py 의 summarize)의 일이고, 화면은 받은 것을 그리기만 한다
----------------	--

③ 화면에 한계 문구가 남아 있는지 확인

기획안 4.4가 “정확한 척하지 않는 것이 이 서비스의 태도”라고 못박았습니다. 화면 맨 아래 이 두 문장이 지워지지 않았는지 발표 전에 확인하세요.

보증하는 것 / 보증하지 않는 것

보증하는 것 — 기록이 작성된 뒤 고쳐지지 않았다는 사실.

보증하지 않는 것 — 본인이 직접 작업했는지 여부. AI 사용 여부를 판정하지 않습니다. 어떤 파일을 한 작품으로 묶을지는 작성자가 정합니다. 해석은 보는 사람의 몫입니다.

④ 인터뷰 8~10명

기획안 8.3 표를 실제 이름으로 채우는 일입니다. 특히 커미션 작가 3명 섭외 경로를 회의 전에 정해 오세요.

대상	인원	핵심 질문
커미션 작가 (X·크몽)	3	AI 의심을 받은 적 있나. 그때 뭘 보냈나. 500원이면 내겠나
교양 글쓰기 담당 교수	1	탐지기를 쓰나. 오탐 소명은 어떻게 처리하나
오탐 피해 학생	1	그때 무엇이 있었으면 소명이 됐을까
AI를 공개적으로 쓰는 창작자	1	“AI 초안 + 편집” 증명서도 쓸 의향이 있나
커미션 의뢰인	1	증명서가 있으면 더 내겠나
또래 (대조군)	2	과제에 스스로 켄 의향이 있나. 감시로 느끼나

⑤ 데스크 크리틱 3문장 + 리허설

기획안 11.7 입니다. 발표는 아래 네 컷으로 갑니다. 순서대로 손에 익혀 두세요.

1 작성 영역에서 글을 쓰고 저장한다

개발자도구(F12) Network 탭을 열어 두면 snapshot 요청이 흐르는 게 보입니다.

2 화면 위 초록 — “이 기록은 변조되지 않았습니다”

3 data\chain.json 을 VS Code 로 열어 아무 content_hash 의 첫 글자를 바꾸고 저장

발표 전에 어느 줄을 고칠지 미리 정해 두세요. 데모 3보다 데모 2가 스냅샷이 적어서 찾기 쉽습니다.

4 “다시 검증”을 누른다 → 빨강

어느 파일의 몇 번 스냅샷에서 끊겼는지까지 화면에 나옵니다.

chain.json 은 메모장으로 열지 마세요

윈도우 메모장은 저장할 때 파일 앞에 눈에 안 보이는 표시(BOM)를 붙이거나 줄바꿈을 바꿀 수 있습니다. 그러면 서버가 “chain.json 을 읽을 수 없습니다” 라고 하면서 화면이 통째로 비어 버립니다. 조작 시연이 아니라 그냥 고장이 나는 거라 발표에서 곤란해집니다.

VS Code 로 여세요. 오른쪽 아래에 UTF-8 이라고 떠 있으면 안전합니다. 이미 망가뜨렸으면 앞 장의 되돌리기 명령어를 쓰면 됩니다.

이 네 것이 “데이터 흐름”과 “사후 조작 불가”를 한 장면에 담습니다.

5. 회의에서 정할 것

A 혼자 정할 수 없는 것들입니다. 읽고 의견을 정해서 오세요.

① 작성자 자기신고를 넣을 것인가 — 기획안에 없던 기능

붙여넣기 직후에 “어디서 가져왔나요?”를 묻고 (AI 초안 / 인용 / 내 예전 원고 / 표시 안 함), 그 답을 체인에 묶어 나중에 못 고치게 하는 기능입니다.

찬성 근거 — “무엇이 AI인가”를 서비스가 판정하지 않으면서 보는 사람의 질문에 답하는 유일한 길입니다. 기획안 4.3의 “AI 초안을 붙여넣고 편집 → 드러남 → AI 초안 + 편집도 정직한 증명서”를 화면으로 옮긴 것입니다.

반대 근거 — 기획안에 없던 기능이라 범위가 늘어납니다. 화면 설계에는 이미 들어가 있어서, 여기가 없으면 검증 화면 문구도 같이 바뀝니다.

② 체인 검증을 기획안에 맞출 것인가

기획안 4.2는 “중간 스냅샷 하나만 바뀌도 이후가 전부 어긋난다”고 되어 있는데, 지금 코드는 고친 그 한 줄만 깨집니다.

```
prev = snap["hash"]
```

다시 계산한 값(expected)이 아니라 파일에 적힌 값을 이전 해시로 쓰기 때문입니다.

이 한 줄을 `prev = expected` 로 바꾸면 기획안대로 됩니다. 지금 화면 설계는 코드 쪽(한 줄만 깨짐)에 맞춰 그려 뒀습니다. 발표에서 질문받기 쉬운 지점이라 어느 쪽으로 통일할지 정해야 합니다.

③ 외부 앵커를 이번 프로토타입에 넣을 것인가

넣지 않기를 권합니다. 네트워크 변수 때문에 시연 중 깨질 위험이 있습니다. 로드맵에만 두고 화면에는 자리만 비워 두는 쪽입니다. 다만 앵커가 없으면 “위조자가 이후 해시를 전부 다시 계산하면?”이라는 질문에 시연으로 답할 수 없다는 점은 알고 있어야 합니다.

이 밖에 기획안 전문과 PPT의 가격표가 서로 다릅니다. 발표 전에 통일해야 합니다. (증명서 1건 500원 / 1,000원, 작가 무제한 월 4,900원 / 8,900원, 기관 좌석 연 1,000원 / 1인 10,000원)

6. 자주 쓸 것

전부 Anaconda Prompt 에서, 먼저 아래 두 줄을 치고 시작합니다.

```
conda activate cp
```

```
cd %USERPROFILE%\dev\trace
```

주소 · 명령어

서버 켜기	python app.py → 브라우저에 127.0.0.1:5001 (끄기는 Ctrl+C)
데모 다시 만들기	python seed.py
작업 올리기	git add -A → git commit -m "... " → git push
A가 올린 것 받기	git pull
저장소	github.com/20233014-KYH/trace
화면 설계 10장	claude.ai/code/artifact/2a6bae09-b4f8-4e3f-913d-6acba501ebed
바뀐 점 설명	저장소의 CHANGES.md
프로젝트 전반	저장소의 HANDOFF.md

화면 설계와 코드는 일부러 다릅니다

화면 설계는 쓰던 도구 위에 떠 있는 상주 프로그램(수집기 B, 로드맵 W10 이후)을 그린 것이고, 이번 주 프로토타입은 강의자료 STEP 1 범위라 웹 에디터로 갑니다.

발표에서는 이렇게 말하면 됩니다 — “프로토타입은 웹 에디터로 데이터 흐름을 증명하고, 제품은 툴 무관 수집기로 간다.” 기획안 4.1에 수집기 A(웹)와 B(데스크톱)가 이미 나뉘어 있습니다.

막히면 혼자 붙잡지 말고 A에게 연락하세요. 검은 창을 통째로 캡처해서 보내 주면 제일 빠릅니다 — 친 명령어와 오류 메시지가 같이 보여야 합니다.